

MC714 - Sistemas Distribuidos

slides by Maarten van Steen
(adapted from Distributed System - 3rd Edition)

Chapter 07: Consistency & Replication

Version: June 2, 2022

Performance and scalability

Main issue

To keep replicas consistent, we generally need to ensure that all **conflicting** operations are done in the the same order everywhere

Conflicting operations: From the world of transactions

- **Read–write conflict**: a read operation and a write operation act concurrently
- **Write–write conflict**: two concurrent write operations

Issue

Guaranteeing global ordering on conflicting operations may be a costly operation, downgrading scalability **Solution**: weaken consistency requirements so that hopefully global synchronization can be avoided

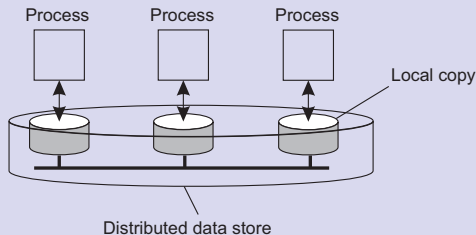
Data-centric consistency models

Consistency model

A contract between a (distributed) data store and processes, in which the data store specifies precisely what the results of read and write operations are in the presence of concurrency.

Essential

A data store is a distributed collection of storages:



Continuous Consistency

We can actually talk about a **degree of consistency**

- replicas may differ in their **numerical value**
- replicas may differ in their relative **staleness**
- there may be differences with respect to (number and order) of **performed update operations**

Conit

Consistency unit $\Rightarrow$ specifies the **data unit** over which consistency is to be measured.

Sequential consistency

Definition

The result of any execution is the same as if the operations of all processes were executed in some sequential order, and the operations of each individual process appear in this sequence in the order specified by its program.

(a) A sequentially consistent data store. (b) A data store that is not sequentially consistent

P1:	W(x)a		
P2:	W(x)b		
P3:		R(x)b	R(x)a
P4:		R(x)b	R(x)a

(a)

P1:	W(x)a		
P2:	W(x)b		
P3:		R(x)b	R(x)a
P4:		R(x)a	R(x)b

(b)

Causal consistency

Definition

Writes that are potentially causally related must be seen by all processes in the same order. Concurrent writes may be seen in a different order by different processes.

(a) A violation of a causally-consistent store. (b) A correct sequence of events in a causally-consistent store

P1:	W(x)a			
P2:	R(x)a	W(x)b		
P3:			R(x)b	R(x)a
P4:			R(x)a	R(x)b

(a)

P1:	W(x)a			
P2:		W(x)b		
P3:			R(x)b	R(x)a
P4:			R(x)a	R(x)b

(b)

Grouping operations

Definition

- Accesses to **locks** are sequentially consistent.
- No access to a lock is allowed to be performed until all previous writes have completed everywhere.
- No data access is allowed to be performed until all previous accesses to locks have been performed.

Grouping operations

Definition

- Accesses to **locks** are sequentially consistent.
- No access to a lock is allowed to be performed until all previous writes have completed everywhere.
- No data access is allowed to be performed until all previous accesses to locks have been performed.

Basic idea

You don't care that reads and writes of a **series** of operations are immediately known to other processes. You just want the **effect** of the series itself to be known.

Grouping operations

A valid event sequence for entry consistency

P1:	L(x) W(x)a L(y) W(y)b U(x) U(y)		
P2:		L(x) R(x)a	R(y) NIL
P3:		L(y) R(y)b	

Observation

Entry consistency implies that we need to lock and unlock data (implicitly or not).

Question

What would be a convenient way of making this consistency more or less transparent to programmers?

Consistency for mobile users

Example

Consider a distributed database to which you have access through your notebook. Assume your notebook acts as a front end to the database.

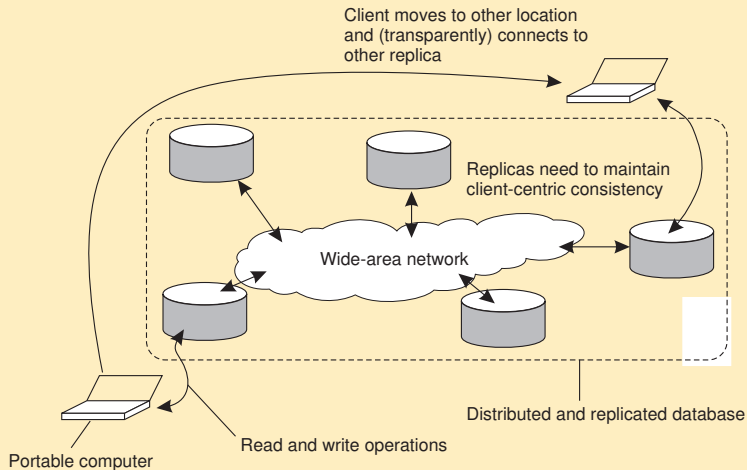
- At location *A* you access the database doing reads and updates.
- At location *B* you continue your work, but unless you access the same server as the one at location *A*, you may detect inconsistencies:
 - your updates at *A* may not have yet been propagated to *B*
 - you may be reading newer entries than the ones available at *A*
 - your updates at *B* may eventually conflict with those at *A*

Note

The only thing you really want is that the entries you updated and/or read at *A*, are in *B* the way you left them in *A*. In that case, the database will appear to be consistent **to you**.

Basic architecture

The principle of a mobile user accessing different replicas of a distributed database



Monotonic reads

Definition

If a process reads the value of a data item x , any successive read operation on x by that process will always return that same or a more recent value.

The read operations performed by a single process P at two different local copies of the same data store. (a) A monotonic-read consistent data store. (b) A data store that does not provide monotonic reads

L1:	$W_1(x_1)$	$R_1(x_1)$
L2:	$W_2(x_1; x_2)$	$R_1(x_2)$

L1:	$W_1(x_1)$	$R_1(x_1)$
L2:	$W_2(x_1 x_2)$	$R_1(x_2)$

Client-centric consistency: notation

Notation

- $W_1(x_2)$ is the write operation by process P_1 that leads to version x_2 of x
- $W_1(x_i; x_j)$ indicates P_1 produces version x_j based on a previous version x_i .
- $W_1(x_i|x_j)$ indicates P_1 produces version x_j **concurrently** to version x_i .

Monotonic reads

Example

Automatically reading your personal calendar updates from different servers. Monotonic Reads guarantees that the user sees all updates, no matter from which server the automatic reading takes place.

Example

Reading (not modifying) incoming mail while you are on the move. Each time you connect to a different e-mail server, that server fetches (at least) all the updates from the server you previously visited.

Monotonic writes

Definition

A write operation by a process on a data item x is completed before any successive write operation on x by the same process.

(a) A monotonic-write consistent data store. (b) A data store that does not provide monotonic-write consistency. (c) Again, no consistency as $WS(x_1|x_2)$ and thus also $WS(x_1|x_3)$. (d) Consistent as $WS(x_1;x_3)$ although x_1 has apparently overwritten x_2 .

L1:	$W_1(x_1)$	
L2:	$W_2(x_1;x_2)$	$W_1(x_2;x_3)$

(a)

L1:	$W_1(x_1)$	
L2:	$W_2(x_1 x_2)$	$W_1(x_1 x_3)$

(b)

L1:	$W_1(x_1)$	
L2:	$W_2(x_1 x_2)$	$W_1(x_2;x_3)$

(c)

L1:	$W_1(x_1)$	
L2:	$W_2(x_1 x_2)$	$W_1(x_1;x_3)$

(d)

Read your writes

Definition

The effect of a write operation by a process on data item x , will always be seen by a successive read operation on x by the same process.

(a) A data store that provides read-your-writes consistency. (b) A data store that does not.

L1:	$W_1(x_1)$	
L2:	$W_2(x_1; x_2)$	$R_1(x_2)$

(a)

L1:	$W_1(x_1)$	
L2:	$W_2(x_1 x_2)$	$R_1(x_2)$

(b)

Read your writes

Definition

The effect of a write operation by a process on data item x , will always be seen by a successive read operation on x by the same process.

(a) A data store that provides read-your-writes consistency. (b) A data store that does not.

L1:	$W_1(x_1)$	
L2:	$W_2(x_1; x_2)$	$R_1(x_2)$

(a)

L1:	$W_1(x_1)$	
L2:	$W_2(x_1 x_2)$	$R_1(x_2)$

(b)

Example

Updating your Web page and guaranteeing that your Web browser shows the newest version instead of its cached copy.

Writes follow reads

Definition

A write operation by a process on a data item x following a previous read operation on x by the same process, is guaranteed to take place on the same or a more recent value of x that was read.

(a) A writes-follow-reads consistent data store. (b) A data store that does not provide writes-follow-reads consistency

L1:	$W_1(x_1)$	$R_2(x_1)$
L2:	$W_3(x_1; x_2)$	$W_2(x_2; x_3)$

(a)

L1:	$W_1(x_1)$	$R_2(x_1)$
L2:	$W_3(x_1 x_2)$	$W_2(x_1 x_3)$

(b)

Example

See reactions to posted articles only if you have the original posting (a read “pulls in” the corresponding write operation).

Replica placement

Essence

Figure out what the best K places are out of N possible locations.

Replica placement

Essence

Figure out what the best K places are out of N possible locations.

- Select best location out of $N - k$ for which the **average distance to clients is minimal**. Then choose the next best server. (**Note:** The first chosen location minimizes the average distance to all clients.) **Computationally expensive**.

Replica placement

Essence

Figure out what the best K places are out of N possible locations.

- Select best location out of $N - k$ for which the **average distance to clients is minimal**. Then choose the next best server. (**Note:** The first chosen location minimizes the average distance to all clients.) **Computationally expensive**.
- Select the K -th largest **autonomous system** and place a server at the best-connected host. **Computationally expensive**.

Replica placement

Essence

Figure out what the best K places are out of N possible locations.

- Select best location out of $N - k$ for which the **average distance to clients is minimal**. Then choose the next best server. (**Note:** The first chosen location minimizes the average distance to all clients.) **Computationally expensive**.
- Select the K -th largest **autonomous system** and place a server at the best-connected host. **Computationally expensive**.
- Position nodes in a d -dimensional geometric space, where distance reflects latency. Identify the K regions with highest density and place a server in every one. **Computationally cheap**.

Content replication

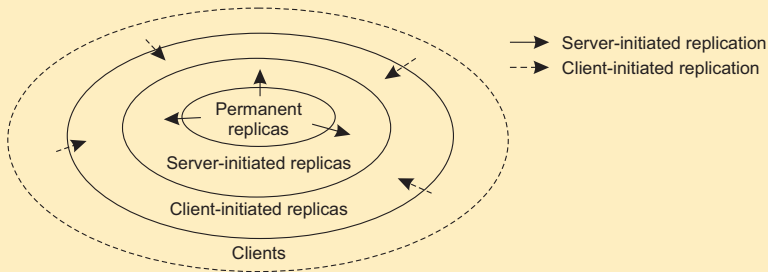
Distinguish different processes

A process is capable of hosting a replica of an object or data:

- **Permanent replicas:** Process/machine always having a replica
- **Server-initiated replica:** Process that can dynamically host a replica on request of another server in the data store
- **Client-initiated replica:** Process that can dynamically host a replica on request of a client (**client cache**)

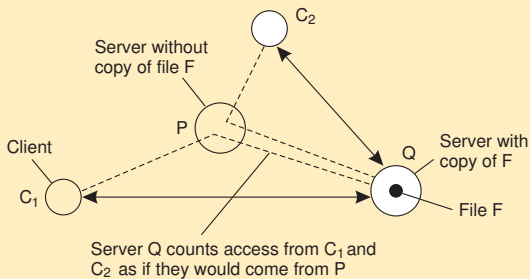
Content replication

The logical organization of different kinds of copies of a data store into three concentric rings



Server-initiated replicas

Counting access requests from different clients



- Keep track of access counts per file, aggregated by considering server closest to requesting clients
- Number of accesses drops below threshold $D \Rightarrow$ drop file
- Number of accesses exceeds threshold $R \Rightarrow$ replicate file
- Number of access between D and $R \Rightarrow$ migrate file

Client-initiated replicas

- Commonly known as (client) caches.
- Managing the cache is left to the client → occasionally the client can rely on the data store to inform it when data has become stale.
- Improve access times to data.
- Located on the client's machine or on a machine in the same LAN.
- Caches can be shared by clients to improve cache hits.
- Data is kept in a cache for a limited amount of time.

Content distribution

Consider only a client-server combination

- Propagate only **notification/invalidation** of update (often used for caches)
- Transfer **data** from one copy to another (distributed databases): **passive replication**
- Propagate the update **operation** to other copies: **active replication**

Note

No single approach is the best, but depends highly on available bandwidth and read-to-write ratio at replicas.

Content distribution: client/server system

A comparison between push-based and pull-based protocols in the case of multiple-client, single-server systems

- **Pushing updates:** **server-initiated approach**, in which update is propagated regardless whether target asked for it.
- **Pulling updates:** **client-initiated approach**, in which client requests to be updated.

Issue	Push-based	Pull-based
1:	List of client caches	None
2:	Update (and possibly fetch update)	Poll and update
3:	Immediate (or fetch-update time)	Fetch-update time
1: <i>State at server</i>		
2: <i>Messages to be exchanged</i>		
3: <i>Response time at the client</i>		

Content distribution

Observation

We can dynamically switch between pulling and pushing using **leases**: A contract in which the server promises to push updates to the client until the lease expires.

Make lease expiration time dependent on system's behavior (adaptive leases)

Content distribution

Observation

We can dynamically switch between pulling and pushing using **leases**: A contract in which the server promises to push updates to the client until the lease expires.

Make lease expiration time dependent on system's behavior (adaptive leases)

- **Age-based leases**: An object that hasn't changed for a long time, will not change in the near future, so provide a long-lasting lease

Content distribution

Observation

We can dynamically switch between pulling and pushing using **leases**: A contract in which the server promises to push updates to the client until the lease expires.

Make lease expiration time dependent on system's behavior (adaptive leases)

- **Renewal-frequency based leases**: The more often a client requests a specific object, the longer the expiration time for that client (for that object) will be

Content distribution

Observation

We can dynamically switch between pulling and pushing using **leases**: A contract in which the server promises to push updates to the client until the lease expires.

Make lease expiration time dependent on system's behavior (adaptive leases)

- **State-based leases**: The more loaded a server is, the shorter the expiration times become

Content distribution

Observation

We can dynamically switch between pulling and pushing using **leases**: A contract in which the server promises to push updates to the client until the lease expires.

Make lease expiration time dependent on system's behavior (adaptive leases)

- **Age-based leases**: An object that hasn't changed for a long time, will not change in the near future, so provide a long-lasting lease
- **Renewal-frequency based leases**: The more often a client requests a specific object, the longer the expiration time for that client (for that object) will be
- **State-based leases**: The more loaded a server is, the shorter the expiration times become

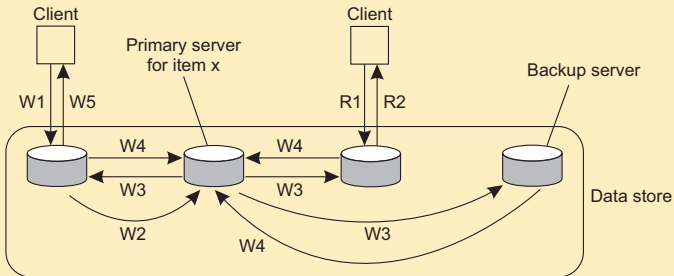
Question

Why are we doing all this?

Primary-based protocols

*sequential consistency

Primary-backup protocol

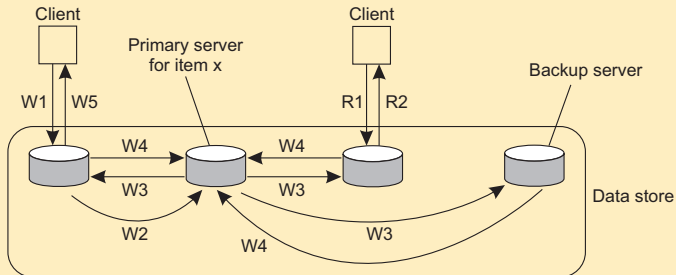


W1. Write request
 W2. Forward request to primary
 W3. Tell backups to update
 W4. Acknowledge update
 W5. Acknowledge write completed

R1. Read request
 R2. Response to read

Primary-based protocols

Primary-backup protocol



W1. Write request
W2. Forward request to primary
W3. Tell backups to update
W4. Acknowledge update
W5. Acknowledge write completed

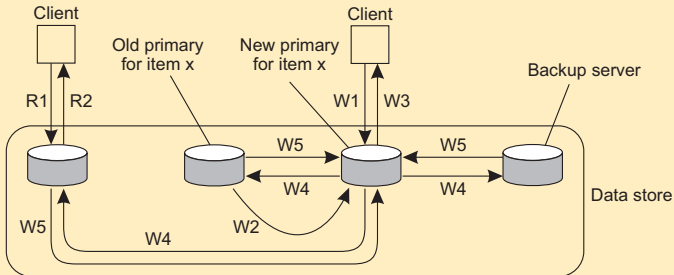
R1. Read request
R2. Response to read

Example primary-backup protocol

Traditionally applied in distributed databases and file systems that require a high degree of fault tolerance. Replicas are often placed on same LAN.

Primary-based protocols

Primary-backup protocol with local writes

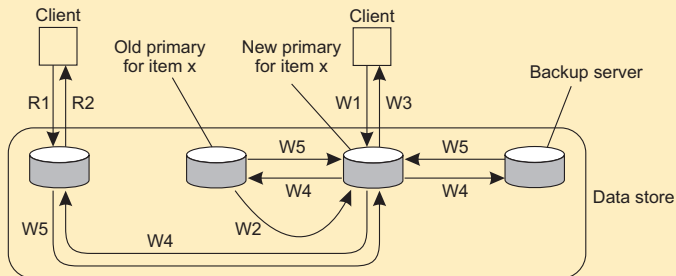


W1. Write request
 W2. Move item x to new primary
 W3. Acknowledge write completed
 W4. Tell backups to update
 W5. Acknowledge update

R1. Read request
 R2. Response to read

Primary-based protocols

Primary-backup protocol with local writes



W1. Write request
 W2. Move item x to new primary
 W3. Acknowledge write completed
 W4. Tell backups to update
 W5. Acknowledge update

R1. Read request
 R2. Response to read

Example primary-backup protocol with local writes

Mobile computing in disconnected mode (ship all relevant files to user before disconnecting, and update later on).

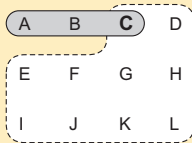
Replicated-write protocols

Quorum-based protocols

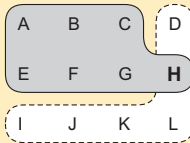
Ensure that each operation is carried out in such a way that a majority vote is established: distinguish **read quorum** and **write quorum**

Three examples of the voting algorithm. (a) A correct choice of read and write set. (b) A choice that may lead to write-write conflicts. (c) A correct choice, known as ROWA (read one, write all)

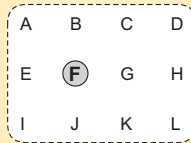
* $N_r + N_w > N$
 $N_w > N/2$



$N_R = 3, N_W = 10$



$N_R = 7, N_W = 6$



$N_R = 1, N_W = 12$